



Macao University of Science and Technology

School of Computer Science and Engineering

Faculty of Innovation Engineering

<Software Engineering Project>

Homework ID: Task1 - Project Proposal

Project Title: Smart Independent Travel Itinerary Planning System

Student Name: Fang Yanbing, Li Mingxuan, Meng Fandi, Yang Ge

Student No.: 1240034597, 1240024624, 1240033061, 1240027561

Date: 2026-10-04

Project Website: <https://github.com/D3-Team4/smart-itinerary-planner>

Team Profile

Fang Yanbing — Project proposal conception; team coordination and management; project progress tracking and control.

Li Mingxuan — Academic writing and final document formatting and logical organization; meeting organization and minute-taking; external interviews to elicit customer requirements.

Meng Fandi — Database design and SQL operations; data preprocessing; project report presentation.

Yang Ge — Programming and functional design; testing and troubleshooting; front-end and back-end development; performance optimization.

Every member is expected to take part in all project activities (design, programming, testing, documentation, and presentation); the strengths above indicate where each member leads, not their sole responsibilities.

Team leader: Fang Yanbing — acts as facilitator, organizing meetings and tracking progress.

Abstract

Planning a multi-attraction trip is a common but surprisingly tedious task for independent travelers. Today a traveler who wants to visit several places in one day must decide the order of visits and how to travel between every pair of them, a process that is done almost entirely by hand, with map apps and travel guides, and that rarely produces the most efficient plan. We propose a smart itinerary planning system in which a traveler simply enters the places they want to visit and chooses a preference (fastest, cheapest, or fewest transfers), and the system automatically produces an optimized whole-day plan: the best visiting order together with the best way to travel for each leg. The system relies on a freely available online map and public-transit data service and on a well-known route-optimization algorithm. This proposal describes the problem we address, our proposed solution and its business value, and our plan of work.

Table of Contents

Abstract.....	II
Table of Contents	III
List of Figures	1
List of Tables	2
Chapter 1	3
1. Introduction.....	3
1.1 Background	3
1.2 Problem Diagnosis	3
1.3 Project Goals.....	4
2. Proposed Project.....	4
2.1 Project Overview	4
2.2 Typical Customers	4
2.3 Functional Features	5
2.4 Solution Overview and Typical Usage Scenario	5
2.5 Value to Users	6
2.6 Success Metrics	6
2.7 Required External Resources	6
3. Plan of Work and Product Ownership.....	6
3.1 Phased Work Plan (Coming Weeks).....	7
3.1.1 Phase 1: Preparation and Data Layer	7
3.1.2 Phase 2: Parallel Development of Core Modules	7
3.1.3 Phase 3: Result Presentation and System Integration	7
3.1.4 Phase 4: System Testing and Prototype Delivery	7
3.2 Pair Assignments: Functionality and Qualitative Properties	8
Pair 1 (Back-End and Algorithms, 2 members).....	8
Pair 2 (Front-End and Visualization, 2 members).....	8

Product Ownership and Collaboration	9
Future Work (Out of Scope for This Phase)	9
4. Reference	9

List of Figures

List of Tables

Chapter 1

1. Introduction

1.1 Background

Independent travel has become one of the most popular ways to explore a city. A typical day trip involves visiting several attractions: a visitor to Macao, for example, may want to see the Ruins of St. Paul's, Senado Square, the A-Ma Temple, Macao Tower, and Coloane Village in a single day. The hard part of such a trip is not deciding what to see, but deciding in what order to see it and how to get from one place to the next, given that each leg can be made on foot, by bus, or by taxi, with different times and costs.

1.2 Problem Diagnosis

The problem domain is multi-attraction trip planning for independent travelers. When a person wants to visit several attractions in one city, they face two connected questions: (1) in what order should the attractions be visited, and (2) by what means of transport should each leg between consecutive attractions be made. Today this planning is done almost entirely by hand, which produces four concrete pain points:

- **Planning is extremely time-consuming and too large to solve by hand:** Choosing the visiting order is a permutation, so the number of possible orders grows factorially ($n!$ for n attractions), and each order has many transport combinations.
- **The result is rarely efficient:** people arrange attractions in the order they happen to think of them, which often produces backtracking; the order strongly affects total time and cost, but travelers rarely try enough orderings to find a good one.
- **Preferences cannot be applied easily:** a traveler who wants the fastest day, the cheapest day, or the day with the fewest transfers must essentially redo the whole plan each time.

Existing tools help only partially. Map applications tell a user how to travel between two specific points, but do not plan a route through many points; in addition, their multi-stop optimization is limited to driving or walking and does not support

public transit. Travel blogs and guides offer fixed routes that cannot adapt to a user's own list of places or preferences. The gap our work targets is: automatically computing a good visiting order together with a good transport choice for each leg, according to the traveler's stated preference.

1.3 Project Goals

Our goal is to build a system that reduces the user's manual work to a single step — entering their attractions and picking a preference — and automatically produces a complete, ready-to-follow itinerary. Specifically, we aim to (1) automatically obtain the available transport options between every pair of attractions, (2) compute a good visiting order and a transport choice for each leg that matches the user's preference, and (3) present the result clearly.

2. Proposed Project

2.1 Project Overview

We propose a smart independent travel itinerary planning system. The idea is simple: the user tells the system where they want to go and what they care about (fastest, cheapest, or fewest transfers), and the system does the rest. It automatically gathers the available ways to travel between every pair of attractions, computes a good visiting order and the best transport for each leg, and presents a complete, ready-to-follow itinerary.

2.2 Typical Customers

The core customers of this system are independent travelers — individuals or small groups who do not join guided tours and arrange their own trips. Based on interviews with classmates and friends who have traveled independently, there are four main groups of typical users:

- **Independent day-trippers:** travelers who visit a city for a day or a few days, e.g., a weekend visit to Macao wanting to see five or six attractions, but not knowing in what order to visit them or how to travel between them most conveniently.
- **Young people traveling together:** friends, couples, or roommates who need to balance several people's time, spending, and energy, and want a plan that is both fast and affordable.

- **Business travelers with limited time:** people fitting sightseeing around a business trip or layover, who need a fast, efficient route with little backtracking.
- **Families with elderly members or children:** they care more about fewer transfers and less walking than about pure speed or cost.

2.3 Functional Features

After the project is completed, users will be able to:

- Create a new itinerary and freely add any number of attractions.
- Set a starting point for the itinerary, and optionally an ending point.
- **Set itinerary constraints:** departure time, maximum budget, and maximum total duration.
- **Select an itinerary preference:** fastest, cheapest, or fewest transfers.
- Have the system automatically retrieve walking, public-transport (bus), and taxi options between attractions, including specific routes, travel time, and cost.
- Have the system combine user preferences and custom constraints to automatically compute an optimized visiting order and the transport mode for each leg.
- View the complete itinerary in text form, or inspect attraction points and routes on a simple map.

2.4 Solution Overview and Typical Usage Scenario

All user operations are completed on a single page, in four steps:

1. **Create an itinerary:** enter the attractions to visit, set a starting point (and an optional ending point), and configure the departure time, budget limit, and total duration limit.
2. **Select a preference:** choose among fastest, cheapest, or fewest transfers.
3. **System planning:** the system automatically retrieves the available transport options between attractions and, combining the selected preference with the user constraints, computes an optimized tour and travel plan.
4. **View results:** the complete itinerary is displayed in text form, and attraction points and routes can be inspected on a simple map.

For example, consider a visitor in Macao who wants to see five attractions in one day, such as the Ruins of St. Paul's and Senado Square. The user enters the

attractions, sets the departure time, budget, and total duration limits, selects “fewest transfers,” and within a few seconds receives a complete itinerary.

2.5 Value to Users

Compared with the current manual practice, our system offers clear benefits:

- **Time saved:** what takes up to an hour of manual searching is reduced to entering a list of places and clicking a button; the plan is produced in seconds.
- **Better plans:** the automatically computed route is typically shorter, cheaper, or more convenient than a hand-assembled one.
- **Less decision fatigue:** the system compares the transport options for the user.
- **Easy to change:** switching the preference regenerates the whole plan in one click.

2.6 Success Metrics

We will know the system succeeds when:

- A complete itinerary is generated within a few seconds for a trip of up to eight attractions.
- The generated route's total time and cost are no worse, and usually better, than hand-made plans for the same trips.
- A user can go from 'a list of places' to 'a complete plan' in one short interaction, instead of dozens of separate searches.

2.7 Required External Resources

To accomplish the goal we need two resources, both freely available:

- **Access to a map and public-transit data service:** we will use the Amap Open Platform, a widely used online map service that provides walking, bus, and driving directions with time and cost, free of charge for development use.
- **A route-optimization algorithm:** we plan to adapt the well-known traveling-salesman approach (a standard technique for finding a good order to visit many points); the specific algorithm will be finalized during the design phase.

3. Plan of Work and Product Ownership

Our team consists of four members, organized into two pairs of two. Over the coming weeks, we plan to implement all core features defined in Section 2.3 and deliver a demonstrable system prototype. Advanced enhancements are deferred to

future work and are out of scope for this prototype. All features developed in this phase come from the description in Section 2; no additional user capabilities will be introduced.

3.1 Phased Work Plan (Coming Weeks)

3.1.1 Phase 1: Preparation and Data Layer

Configure the overall development environment and compile basic information for each attraction. Integrate the Amap API to obtain geographic and transport-related data, and cache query results locally to reduce redundant API calls. In parallel, set up the basic Streamlit front-end framework. Both pairs will jointly agree on the front-end/back-end interface specification and data formats.

3.1.2 Phase 2: Parallel Development of Core Modules

Pair 1 builds a time-windowed TSP model with OR-Tools CP-SAT, treating the user-specified departure time, budget ceiling, and total duration as hard constraints; implements the route-optimization logic; adds constraint feasibility checking; and completes unit tests for the module. Pair 2 develops the full user-interaction interface, supporting creation of a new itinerary, addition of attractions, configuration of start/end points, entry of departure time, budget, and total duration, and selection of travel preferences; unit tests are completed as well.

3.1.3 Phase 3: Result Presentation and System Integration

Pair 1 organizes and encapsulates the algorithm output data structures for convenient front-end consumption. Pair 2 implements textual itinerary presentation and integrates the Amap JavaScript Map API for lightweight route visualization. Both pairs conduct front-end/back-end integration testing to connect the end-to-end flow from user input to itinerary generation, yielding an initially usable prototype.

3.1.4 Phase 4: System Testing and Prototype Delivery

Conduct end-to-end system testing, handle abnormal and edge-case inputs, fix major defects, and release a demonstrable system version.

The focus of this phase is to ensure that the complete business workflow runs reliably; fine-grained polishing and advanced extensions are left to subsequent iterations.

3.2 Pair Assignments: Functionality and Qualitative Properties

Pair 1 (Back-End and Algorithms, 2 members)

Functionality (drawn from the feature list in Section 2.3; implementation tools noted):

1. Retrieve walking, bus, and taxi routes between attractions, together with travel time and cost (Feature 5; tool: Amap Open Platform Web API).
2. Perform route optimization with OR-Tools, accepting departure time, budget, and total duration from the front end as hard solver constraints; perform constraint feasibility checking to determine whether the instance is solvable and avoid emitting invalid itineraries; output the visiting order and the travel plan for each leg (Feature 6; tool: OR-Tools CP-SAT).

Qualitative properties:

1. For inputs of at most eight attractions, the system shall produce a feasible itinerary within five seconds.
2. Local caching shall control API call frequency; missing or abnormal transport data shall be handled gracefully.
3. Generated itineraries shall be consistent with realistic daytime urban travel. If user constraints are too tight to admit a feasible plan, the system shall not output an invalid itinerary.

Pair 2 (Front-End and Visualization, 2 members)

Functionality (drawn from the feature list in Section 2.3; implementation tools noted):

1. Develop the interactive interface for creating a new itinerary and freely adding attractions (corresponds to Sec 2.3: create a new itinerary and add attractions; tool: Streamlit).
2. Support setting a trip start point and an optional end point, and provide input controls for departure time, budget ceiling, and total duration (corresponds to Sec 2.3: set start/end points and itinerary constraints; tool: Streamlit).
3. Provide preference selection among fastest, cheapest, and fewest transfers (corresponds to Sec 2.3: select an itinerary preference; tool: Streamlit).

4. Present the itinerary in textual form and display attraction points and routes on a simple map (corresponds to Sec 2.3: view itinerary text and map; tools: Streamlit and Amap JavaScript Map API).

Qualitative properties:

1. The web interface shall be intuitive so that ordinary travelers without programming background can use it effectively.
2. Unreasonable user input shall trigger clear, user-friendly messages; low-level program errors shall not be exposed directly.
3. Front-end/back-end data exchange shall be stable; page loading and map rendering shall remain smooth.

Product Ownership and Collaboration

The two pairs will meet regularly to align on interface details and development progress, and to resolve issues jointly. Each pair is accountable for the features it implements and for the associated system quality requirements. All members participate in system design, coding, testing, and documentation; the division of labor indicates primary ownership, not exclusive responsibility.

Future Work (Out of Scope for This Phase)

- Implement Pareto multi-objective solving to generate multiple differentiated alternative itineraries.
- Introduce 2-opt and Held–Karp dynamic programming as experimental baselines for assessing solution quality.
- Refine error reporting so that the system can identify and explain which attractions cannot be scheduled due to constraint conflicts.
- Explore reinforcement learning for personalized recommendations and dynamic itinerary re-planning.

4. Reference

1. Amap Open Platform. Web Service API documentation. <https://lbs.amap.com/>
2. Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). The Traveling Salesman Problem: A Computational Study. Princeton University Press.

3. Gavalas, D., Konstantopoulos, C., Mastakas, K., & Pantziou, G. (2014). A survey on algorithmic approaches for solving tourist trip design problems. *Journal of Heuristics*, 20(3), 291–328.

